

09 – Processes & threads

What is happening on my computer?

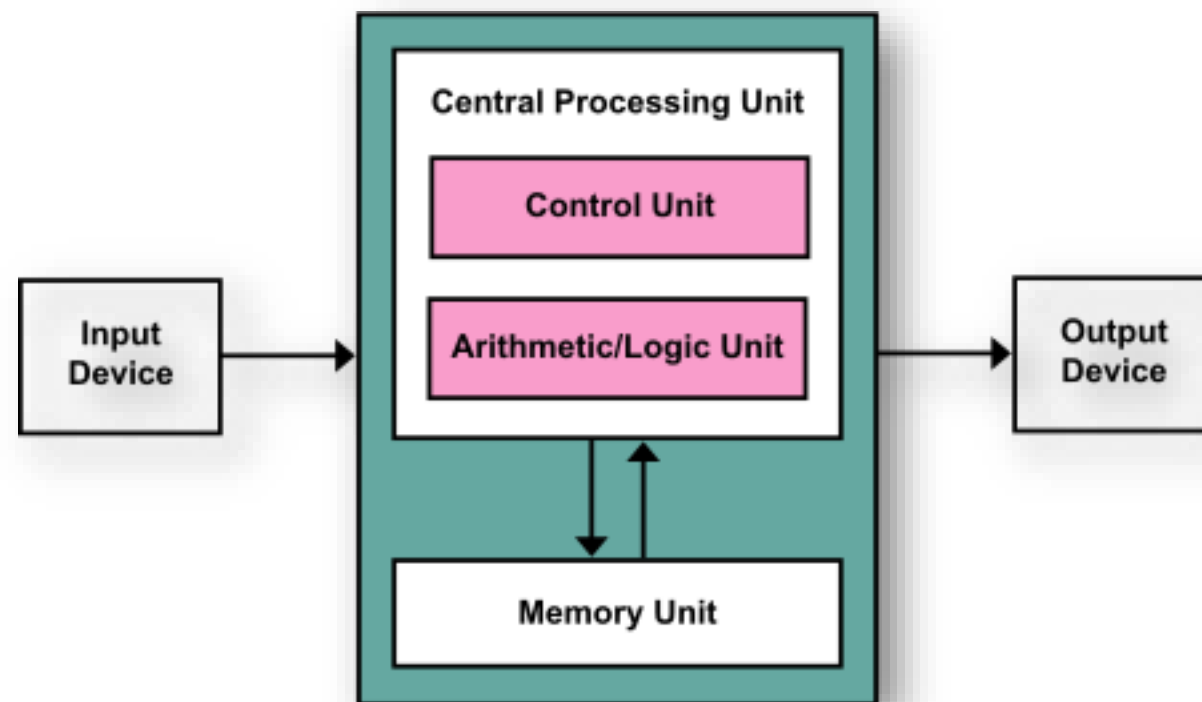
Jérôme Landré – jerome.landre@ju.se

OUTLINE

- 1) Von Neumann Architecture
- 2) Operating Systems
- 3) Process
- 4) Thread
- 5) Comparison between Processes and Threads
- 6) Tree of Processes

1) VON NEUMANN ARCHITECTURE

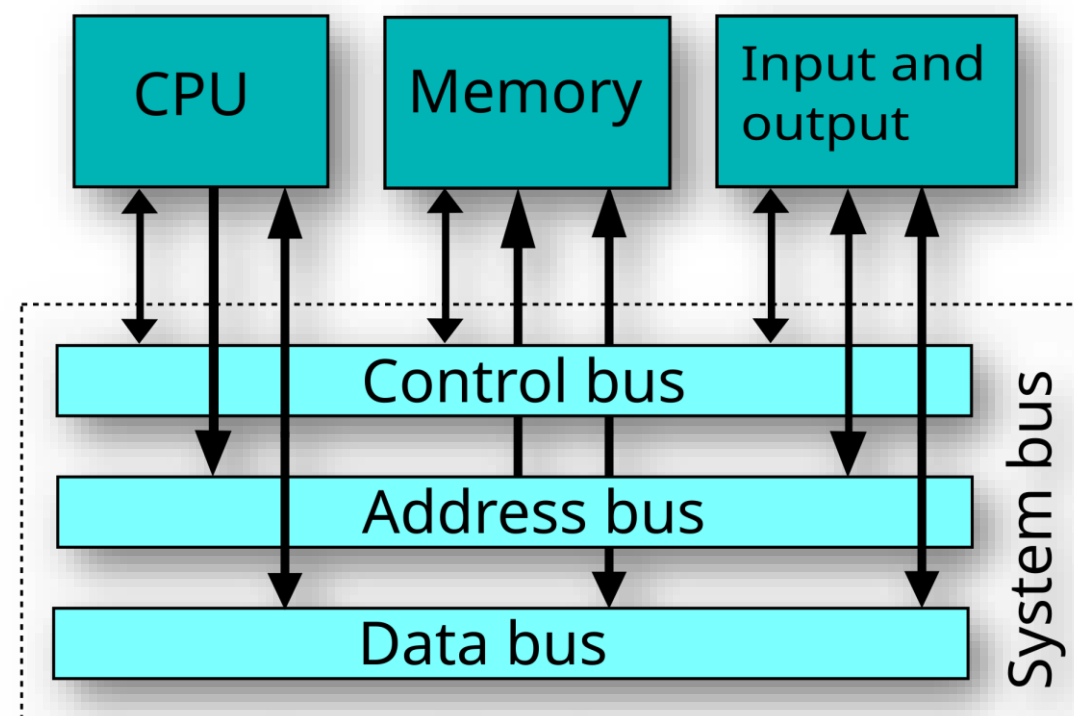
- A computer is composed of:
 - A Central Processing Unit (CPU):
 - A Control Unit (CU)
 - An Arithmetic and Logic Unit (ALU)
 - A Memory Unit (MU)
 - An Input and Output Unit (IOU)



Source: (Wikipedia, "Von Neumann architecture", 2024)

1) VON NEUMANN ARCHITECTURE

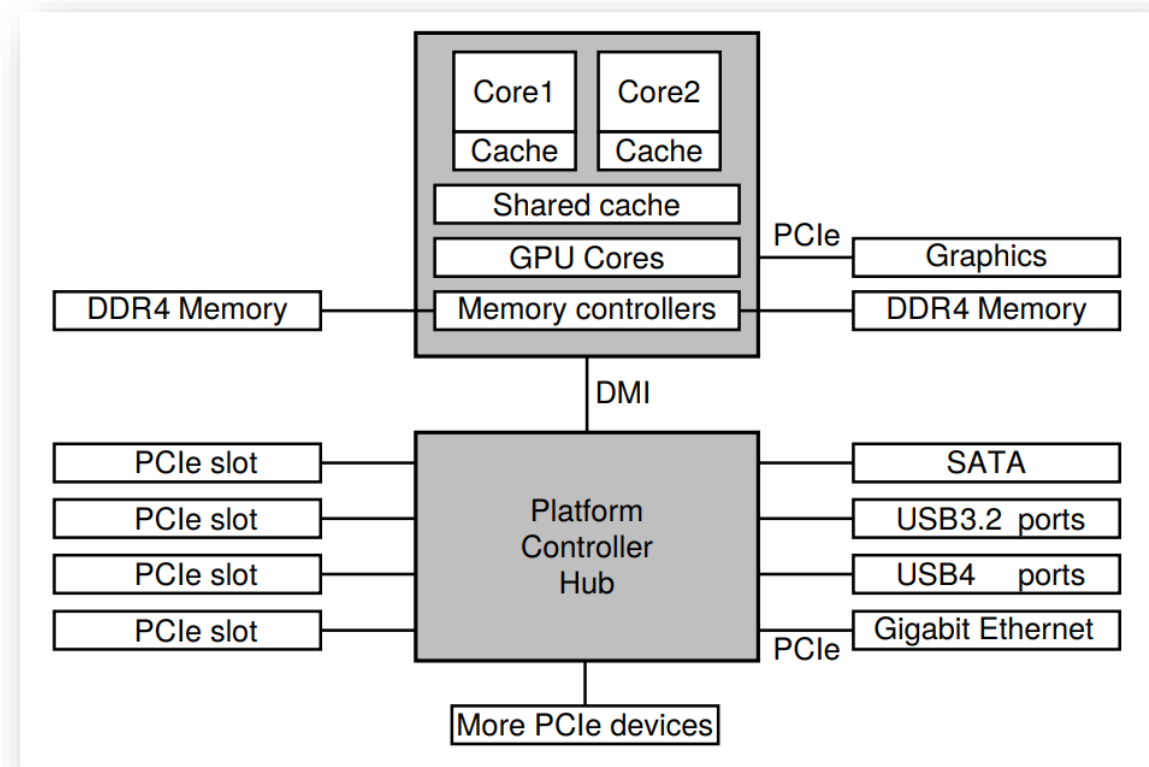
- A computer uses:
 - a Control bus
 - To define the type of operation: read, write, check...
 - An address bus
 - To specify an address
 - A Data bus
 - to receive or send data



1) VON NEUMANN ARCHITECTURE

- A modern computer has a complicated architecture using:

- Multi-core processors
- Cache memory
- Shared cached memory
- GPU cores (PCIe bus)
- Memory controllers
- DMI (Direct Media Interface bus)
- Platform Controller Hub (PCH)
- I/O interfaces: SATA, USB, ...
- Ethernet (PCIe bus)



Source: (Tanenbaum&Bos, "Modern OperatingSystems, fifth edition", Pearson, 2024)

1) VON NEUMANN ARCHITECTURE

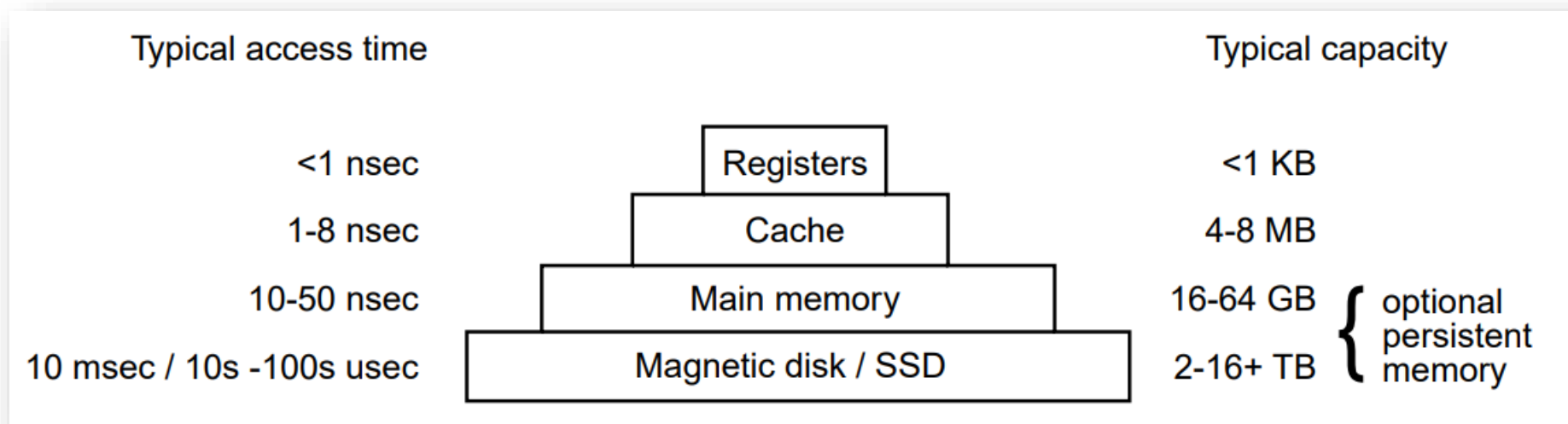
- The CPU is the "brain", the main component of a computer:
 - It extracts instructions from the memory
 - It executes them
 - It has a finite pre-defined set of instructions (**Reduced** in **RISC** architecture)
 - It uses internal registers to accelerate computing

1) VON NEUMANN ARCHITECTURE

- Inside the CPU, registers contain 8, 16, 32, 64 bits memory spaces to store values
- The accumulator (ACC) is the most important register:
 - It stores results from logic and arithmetic operations
- The Program Counter (PC):
 - It stores the address of the next instruction to execute
- The Stack Pointer (SP):
 - It stores the address of the top of the stack (used to store the calls to subprograms)

1) VON NEUMANN ARCHITECTURE

- Accessing memory takes time if it is far away from the CPU
 - **Registers** are **fast**
 - **Disks** are **slow**



Source: (Tanenbaum&Bos, "Modern OperatingSystems, fifth edition", Pearson, 2024)

1) VON NEUMANN ARCHITECTURE

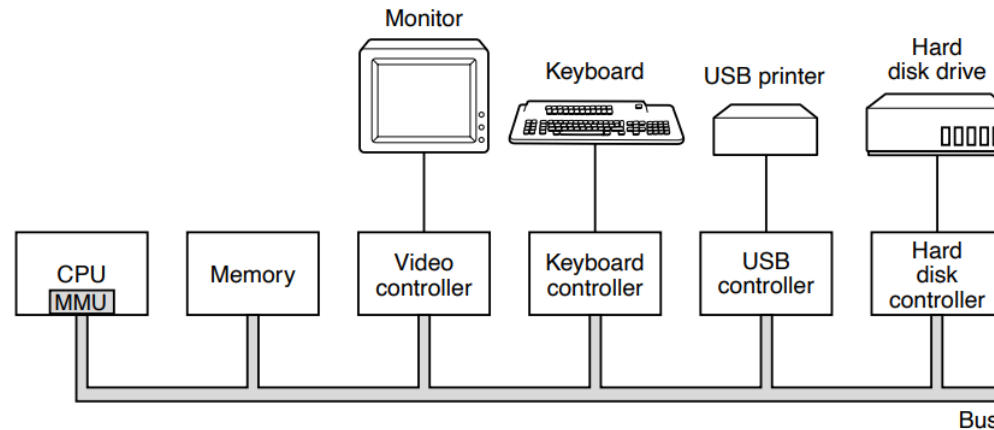


Figure 1-6. Some of the components of a simple personal computer.

Each CPU has a specific set of instructions that it can execute. Thus an x86 processor cannot execute ARM programs and an ARM processor cannot execute x86 programs. Please note that we will use the term **x86** to refer to all the Intel processors descended from the 8088, which was used on the original IBM PC. These include the 286, 386, and Pentium series, as well as the modern Intel core i3, i5, and i7 CPUs (and their clones).

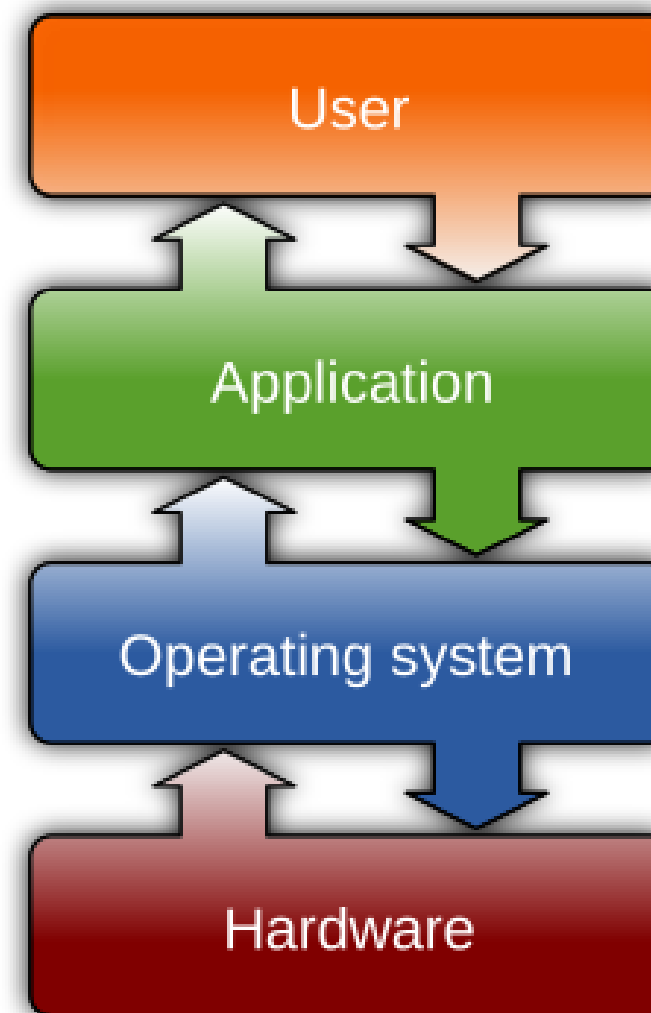
2) OPERATING SYSTEMS

- Operating systems are in between:

the **User/Application**

Operating systems (OS)

and the **Hardware** layers



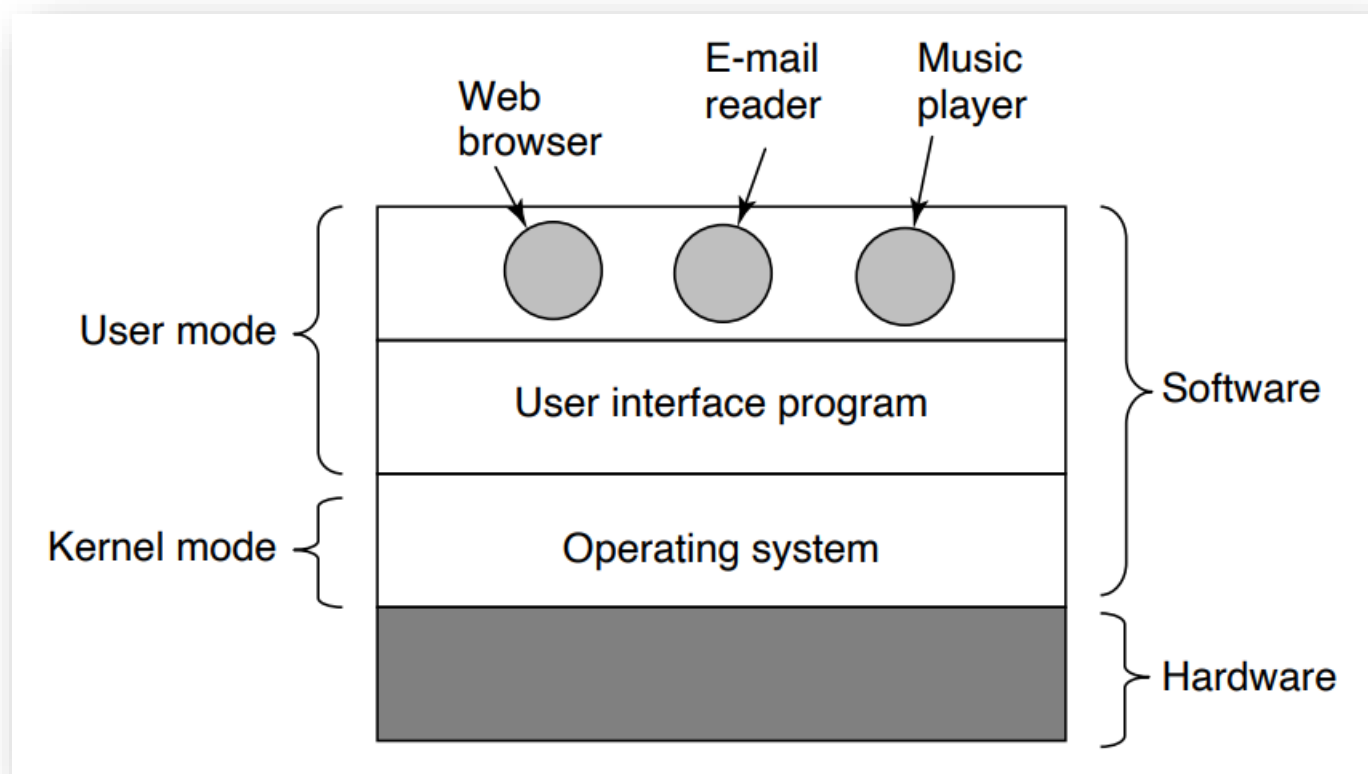
Source: (Wikipedia, " https://en.wikipedia.org/wiki/Memory_management", 2024)

2) OPERATING SYSTEMS

- An OS runs in the **Kernel** mode:
 - Access to all the hardware and software

While

- Applications and user interfaces run in **User** mode
 - Restricted access to hardware and software (according to rights and privileges)



Source: (Tanenbaum&Bos, "Modern OperatingSystems, fifth edition", Pearson, 2024)

2) OPERATING SYSTEMS

- POSIX is a standard for UNIX operating systems created by IEEE:

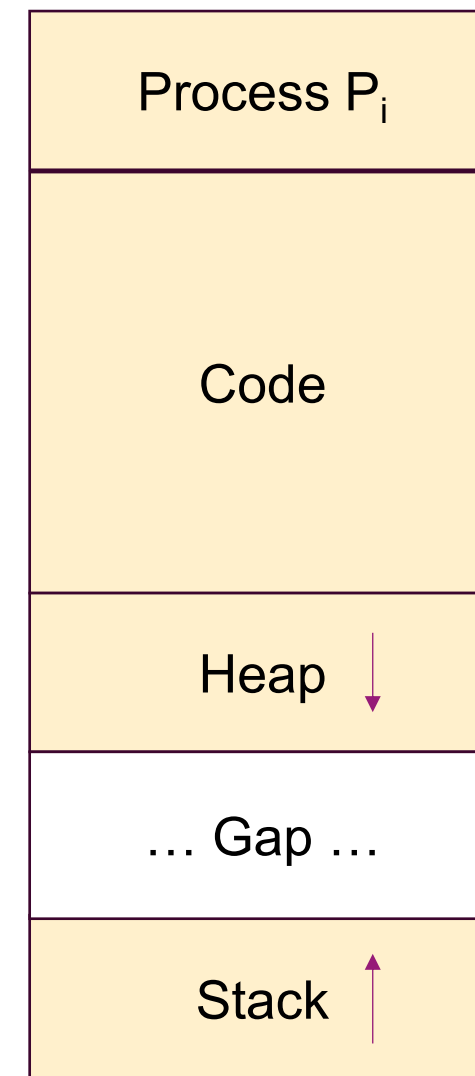
The history of UNIX has been told elsewhere (e.g., Salus, 1994). Part of that story will be given in Chap. 10. For now, suffice it to say that because the source code was widely available, various organizations developed their own (incompatible) versions, which led to chaos. Two major versions developed, **System V**, from AT&T, and **BSD (Berkeley Software Distribution)** from the University of California at Berkeley. These had minor variants as well. To make it possible to write programs that could run on any UNIX system, IEEE developed a standard for UNIX, called **POSIX**, that most versions of UNIX now support. POSIX defines a minimal system-call interface that conformant UNIX systems must support. In fact, some other operating systems now also support the POSIX interface.

3) PROCESS

A key concept in all operating systems is the **process**. A process is basically a program in execution. Associated with each process is its **address space**, a list of memory locations from 0 to some maximum, which the process can read and write. The address space contains the executable program, the program's data, and its stack. Also associated with each process is a set of resources, commonly including registers (including the program counter and stack pointer), a list of open files, outstanding alarms, lists of related processes, and all the other information needed to run the program. A process is fundamentally a container that holds all the information needed to run a program.

3) PROCESS

- A **process** is a program that runs on a computer
- A process P_i is composed of:
 - A **code** to execute (machine language)
 - A **heap** to store the variables and objects of the program
 - A **stack** to store the execution environment



Memory zone allocated
by the OS

3) PROCESS

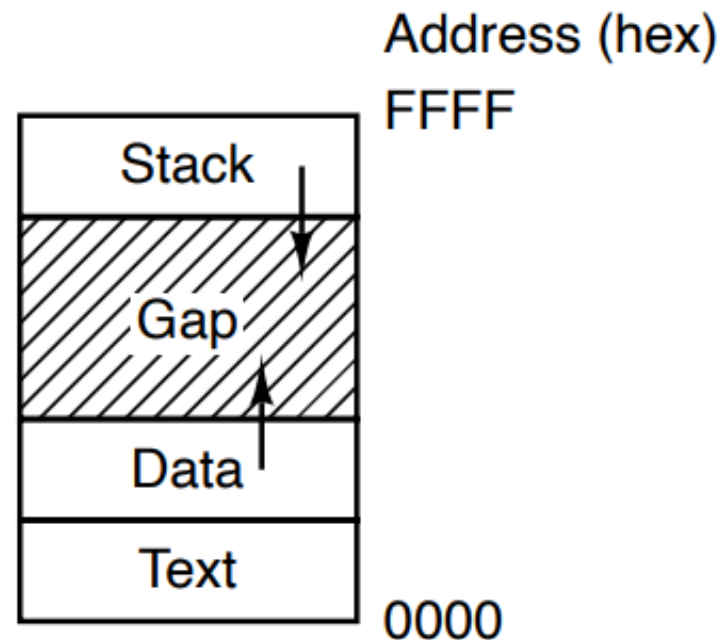


Figure 1-20. Processes have three segments: text, data, and stack.

```
1 #include <stdio.h>
2 #include <unistd.h>
3
4 int my_function(int a, int b); // prototype of the function
5
6 // Uninitialized global variables (stored in BSS)
7 int bss_var;
8 static int static_var;
9
10 // initialized variables
11 int data_var=123;
12
13 int main() {
14     void *stack_var; // uninitialized variable
15     stack_var=(void *) main; // the main program address
16     int main_var=234; // a local variable in main
17
18     printf("Hello world main program is located at %p.\n", stack_var);
19
20     printf("This main_var %d, is located at %p.\n", main_var, &main_var);
21     printf("globalVar: %d, located at %p.\n", bss_var, &bss_var); // Output: 0
22     printf("staticVar: %d, located at %p.\n", static_var, &static_var); // Output: 0
23     printf("Data memory: %d, located at %p.\n", data_var, &data_var); // Output: 123
24     my_function(data_var, main_var);
25     pause();
26     return 0;
27 }
28
29 // a function declared and used in my program
30 int my_function(int a, int b) {
31     int result;
32
33     result=a+b;
34     printf("Result is %d and is located at %p.\n", result, &result);
35     return 0;
36 }
37
```

```
os2025@os2025-virtualbox:~/lecture-processes$ ./a.out
Hello world main program is located at 0x6332e9a5c189.
This main_var 234, is located at 0x7ffd965a79ec.
globalVar: 0, located at 0x6332e9a5f018.
staticVar: 0, located at 0x6332e9a5f01c.
Data memory: 123, located at 0x6332e9a5f010.
Result is 357 and is located at 0x7ffd965a79c4.
```

3) PROCESS

Segment	Description	Example in C
Stack	Local variables, function parameters, return addresses	<code>int x;</code> inside a function
Heap	Dynamically allocated memory (e.g., malloc, calloc)	<code>int *arr = malloc(10 * sizeof(int));</code>
Data Segment	Initialized global and static variables	<code>int globalVar = 42;</code>
BSS Segment	Uninitialized or zero-initialized global and static variables	<code>int uninitializedVar;</code>
Code/Text	Executable instructions (machine code)	The compiled code of <code>main()</code>

3) PROCESS

```
os2025@os2025-virtualbox:~/lecture-processes$ ./a.out
Hello world main program is located at 0x6332e9a5c189.
This main_var 234, is located at 0x7ffd965a79ec.
globalVar: 0, located at 0x6332e9a5f018.
staticVar: 0, located at 0x6332e9a5f01c.
Data memory: 123, located at 0x6332e9a5f010.
Result is 357 and is located at 0x7ffd965a79c4.
```

```
os2025@os2025-virtualbox:~/lecture-processes$ ps aux
USER          PID %CPU %MEM    VSZ   RSS TTY      STAT START   TIME COMMAND
root             1  0.0  0.3  22876 14008 ?        Ss   12:01   0:01 /sbin/init splash
root             2  0.0  0.0      0     0 ?        S    12:01   0:00 [kthreadd]
root             3  0.0  0.0      0     0 ?        S    12:01   0:00 [pool_workqueue_re
lease]
root             4  0.0  0.0      0     0 ?        I<   12:01   0:00 [kworker/R-rcu_gp]
root             5  0.0  0.0      0     0 ?        I<   12:01   0:00 [kworker/R-sync_wa
```

```
root            2232  0.0  0.0      0     0 ?        I    12:34   0:00 [kworker/ds
ee_rcu_reclaim]
os2025          2238  0.0  0.0   2680  1368 pts/1    S+   12:34   0:00 ./a.out
os2025          2239  0.0  0.1  14148  5684 pts/2    Ss   12:34   0:00 /bin/bash
os2025          2245  0.0  0.1  16500  4832 pts/2    R+   12:34   0:00 ps aux
```

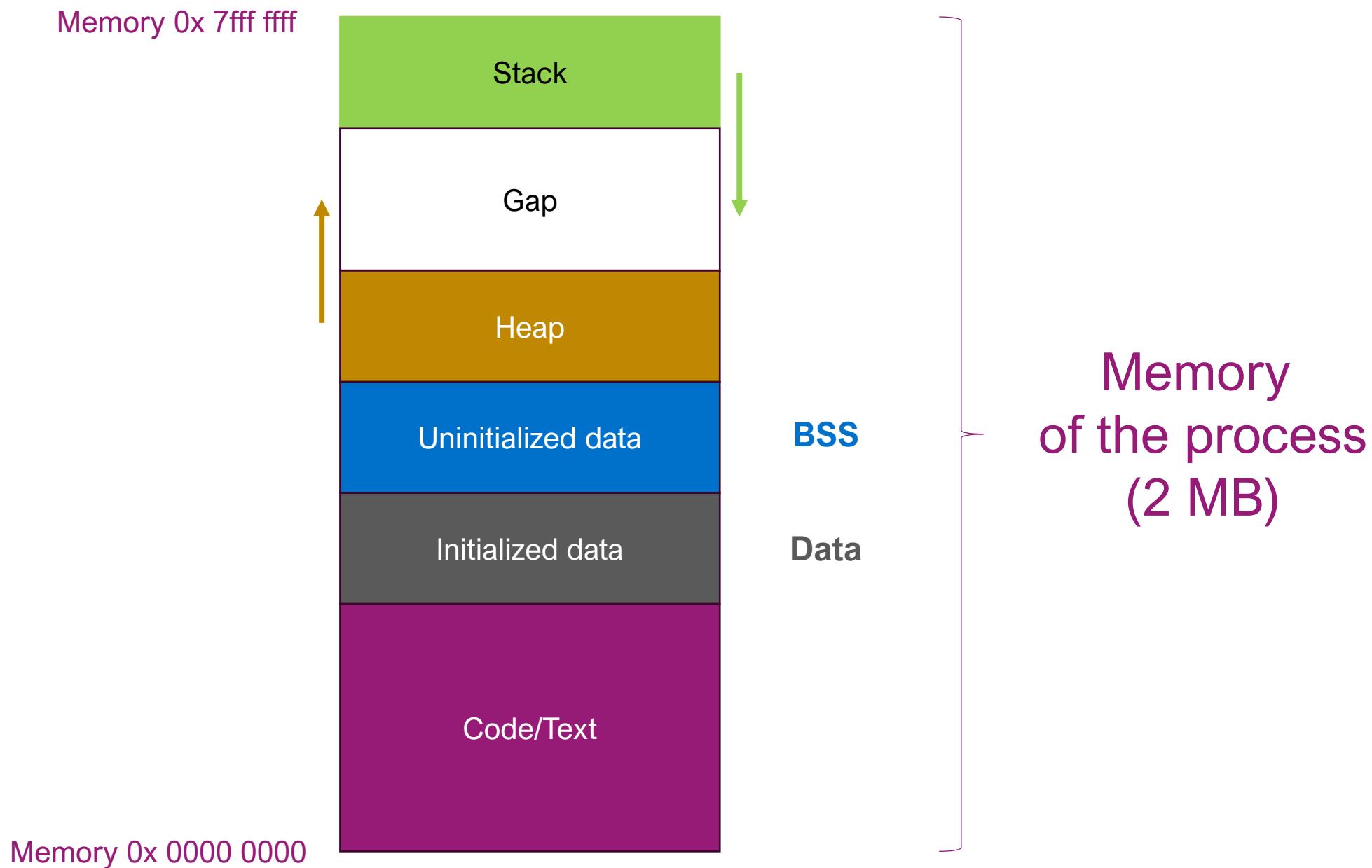
3) PROCESS

```
os2025@os2025-virtualbox:~/lecture-processes$ pmap 2238
2238:  ./a.out
00006332e9a5b000      4K r---- a.out
00006332e9a5c000      4K r-x-- a.out
00006332e9a5d000      4K r---- a.out
00006332e9a5e000      4K r---- a.out
00006332e9a5f000      4K rw--- a.out
0000633310a86000    132K rw--- [ anon ]
00007cda93400000    160K r---- libc.so.6
00007cda93428000   1568K r-x-- libc.so.6
00007cda935b0000    316K r---- libc.so.6
00007cda935ff000     16K r---- libc.so.6
00007cda93603000      8K rw--- libc.so.6
00007cda93605000    52K rw--- [ anon ]
00007cda93760000    12K rw--- [ anon ]
00007cda93776000      8K rw--- [ anon ]
00007cda93778000      8K r---- [ anon ]
00007cda9377a000      8K r---- [ anon ]
00007cda9377c000      8K r-x-- [ anon ]
00007cda9377e000      4K r---- ld-linux-x86-64.so.2
00007cda9377f000   172K r-x-- ld-linux-x86-64.so.2
00007cda937aa000    40K r---- ld-linux-x86-64.so.2
00007cda937b4000      8K r---- ld-linux-x86-64.so.2
00007cda937b6000      8K rw--- ld-linux-x86-64.so.2
00007ffd96588000   132K rw--- [ stack ]
fffffffffff60000      4K --x-- [ anon ]
total                2684K
```

```
os2025@os2025-virtualbox:~/lecture-processes$ strace ./a.out
execve("./a.out", [ "./a.out" ], 0x7fff2ebdb010 /* 59 vars */) = 0
brk(NULL)                               = 0x5b5a617fa000
mmap(NULL, 8192, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_ANONYMOUS, -1, 0) = 0x77d77d50e000
access("/etc/ld.so.preload", R_OK)      = -1 ENOENT (No such file or directory)
openat(AT_FDCWD, "/etc/ld.so.cache", O_RDONLY|O_CLOEXEC) = 3
fstat(3, {st_mode=S_IFREG|0644, st_size=76235, ...}) = 0
mmap(NULL, 76235, PROT_READ, MAP_PRIVATE, 3, 0) = 0x77d77d4fb000
close(3)                                = 0
openat(AT_FDCWD, "/lib/x86_64-linux-gnu/libc.so.6", O_RDONLY|O_CLOEXEC) = 3
read(3, "\177ELF\2\1\1\3\0\0\0\0\0\0\0\0\3\0>\0\1\0\0\0\220\243\2\0\0\0\0"... , 832) = 832
pread64(3, "\6\0\0\0\4\0\0\0@\0\0\0\0\0\0\0@\0\0\0\0\0\0\0@\0\0\0\0\0\0\0"... , 784, 64) = 784
fstat(3, {st_mode=S_IFREG|0755, st_size=2125328, ...}) = 0
pread64(3, "\6\0\0\0\4\0\0\0@\0\0\0\0\0\0\0@\0\0\0\0\0\0\0@\0\0\0\0\0\0\0"... , 784, 64) = 784
mmap(NULL, 2170256, PROT_READ, MAP_PRIVATE|MAP_DENYWRITE, 3, 0) = 0x77d77d200000
mmap(0x77d77d228000, 1605632, PROT_READ|PROT_EXEC, MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0x28000) = 0x77d77d228000
mmap(0x77d77d3b0000, 323584, PROT_READ, MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0x1b0000) = 0x77d77d3b0000
mmap(0x77d77d3ff000, 24576, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0x1fe000) = 0x77d77d3ff000
mmap(0x77d77d405000, 52624, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_FIXED|MAP_ANONYMOUS, -1, 0) = 0x77d77d405000
close(3)                                = 0
mmap(NULL, 12288, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_ANONYMOUS, -1, 0) = 0x77d77d4f8000
arch_prctl(ARCH_SET_FS, 0x77d77d4f8740) = 0
set_tid_address(0x77d77d4f8a10)         = 2311
set_robust_list(0x77d77d4f8a20, 24)     = 0
rseq(0x77d77d4f9060, 0x20, 0, 0x53053053) = 0
mprotect(0x77d77d3ff000, 16384, PROT_READ) = 0
mprotect(0x5b5a3b68f000, 4096, PROT_READ) = 0
mprotect(0x77d77d54c000, 8192, PROT_READ) = 0
prlimit64(0, RLIMIT_STACK, NULL, {rlim_cur=8192*1024, rlim_max=RLIM64_INFINITY}) = 0
munmap(0x77d77d4fb000, 76235)           = 0
fstat(1, {st_mode=S_IFCHR|0620, st_rdev=makedev(0x88, 0x2), ...}) = 0
getrandom("\x87\xd4\x08\x7c\x65\xb7\x71\x42", 8, GRND_NONBLOCK) = 8
brk(NULL)                               = 0x5b5a617fa000
brk(0x5b5a6181b000)                     = 0x5b5a6181b000
write(1, "Hello world main program is loca"... , 55Hello world main program is located at 0x5b5a3b68d169.
) = 55
```

3) PROCESS

```
write(1, "This main_var 234, is located at"... , 49This main_var 234, is located at 0x7ffca033201c.  
) = 49  
write(1, "globalVar: 0, located at 0x5b5a3"... , 41globalVar: 0, located at 0x5b5a3b690018.  
) = 41  
write(1, "staticVar: 0, located at 0x5b5a3"... , 41staticVar: 0, located at 0x5b5a3b69001c.  
) = 41  
write(1, "Data memory: 123, located at 0x5"... , 45Data memory: 123, located at 0x5b5a3b690010.  
) = 45  
write(1, "Result is 357 and is located at "... , 48Result is 357 and is located at 0x7ffca0331ff4.  
) = 48  
exit_group(0)                = ?  
+++ exited with 0 +++  
os2025@os2025-virtualbox:~/lecture-processes$ █
```



3) PROCESS

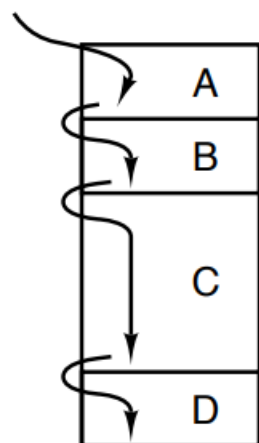
- Processes and threads

In any multiprogramming system, each CPU switches from process to process quickly, running each for tens or maybe hundreds of milliseconds. While strictly speaking, at any one instant each CPU is running only one process, in the course of 1 second it may work on several of them, giving the illusion of parallelism. Sometimes people speak of **pseudoparallelism** in this context, to contrast it with the true hardware parallelism of **multiprocessor** systems (which have two or more CPUs sharing the same physical memory). Keeping track of multiple, parallel activities is hard for people to do. Therefore, operating system designers over the years have evolved a conceptual model (sequential processes) that makes parallelism easier to deal with. That model, its uses, and some of its consequences form the subject of this chapter.

3) PROCESS

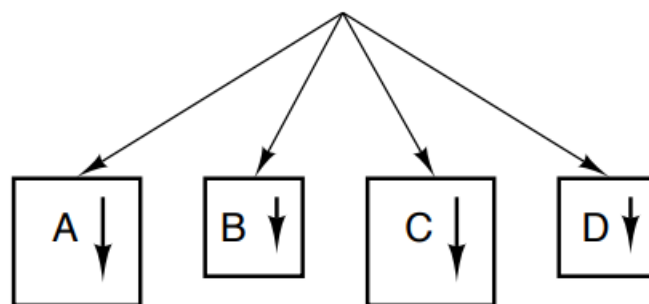
- If you have only one PC (Program Counter) in your CPU, you can only execute the processes in a sequence, one after the other.
- If you have one PC (Program Counter) per process, you can execute the different processes in a concurrent time, sharing the CPU at different times.

One program counter

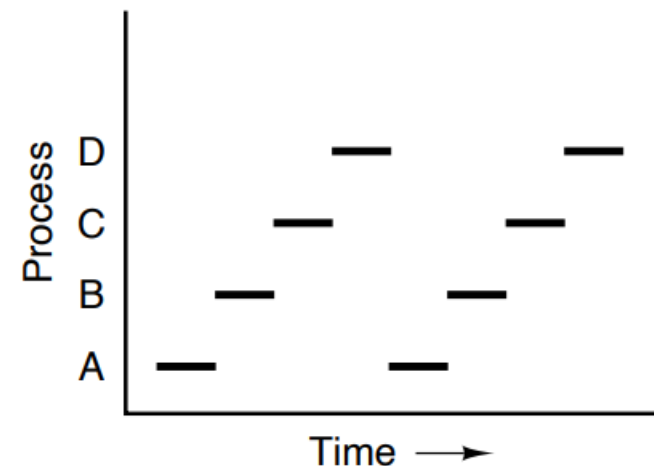


(a)

Four program counters



(b)



(c)

3) PROCESS

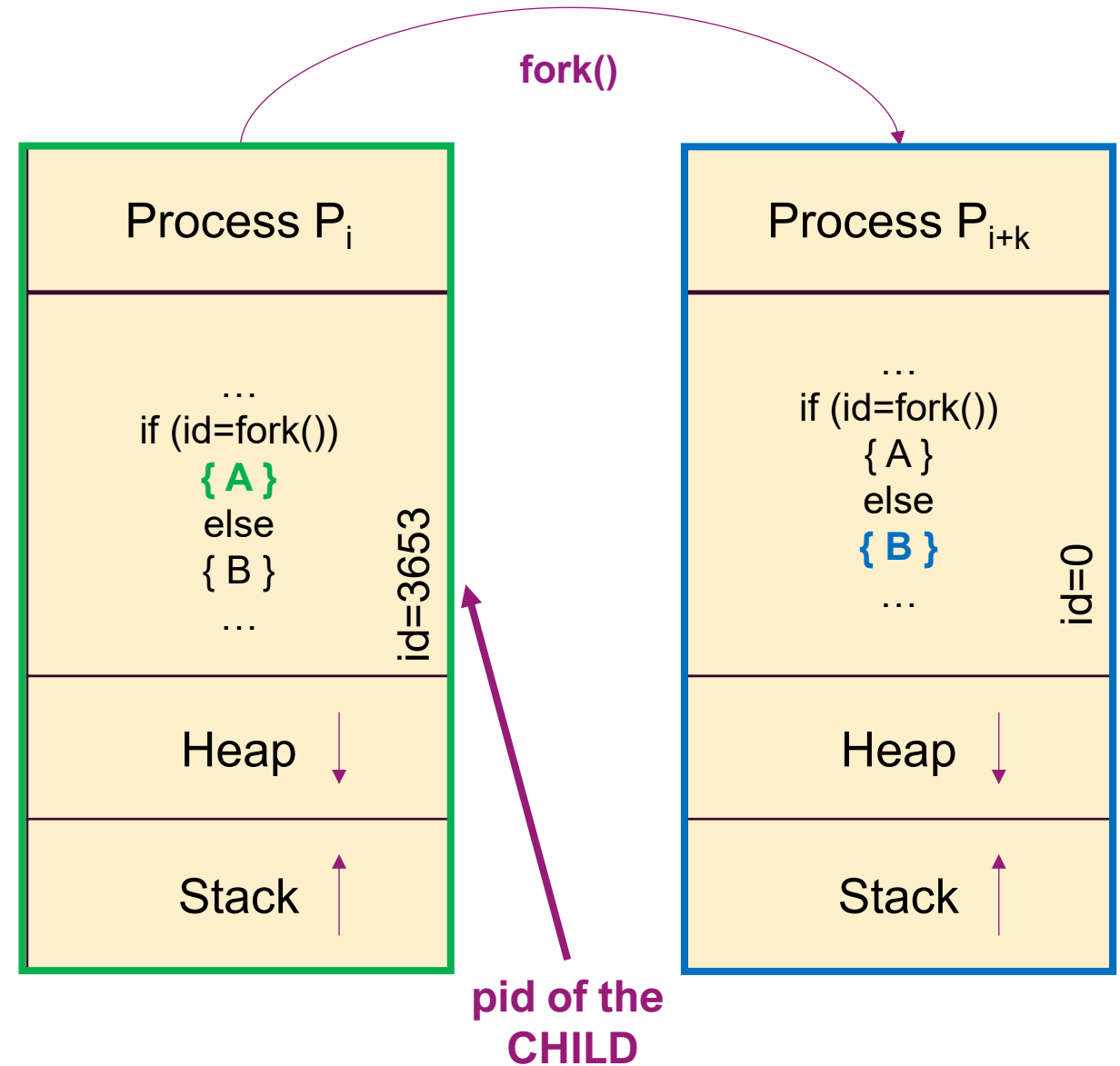
- Creation of a process
 - On Linux, a process is created by running the system function `fork()`
 - `fork()` creates two copies of the running process by copying its code

```
1 #include <stdio.h>
2 #include <unistd.h>
3 #include <sys/types.h>
4
5 int main() {
6     pid_t id=0;
7     printf("Hej!\n");
8     if (id=fork()) {
9         printf("Father process: %d\n", id);
10    } else {
11        printf("Son process: %d\n", id);
12    }
13    return 0;
14 }
15
```

```
[jlandre@jlandre-LINUX code]$ gcc -o fork-03 fork-03.c
[jlandre@jlandre-LINUX code]$ ./fork-03
Hej!
Father process: 3653
Son process: 0
[jlandre@jlandre-LINUX code]$
```

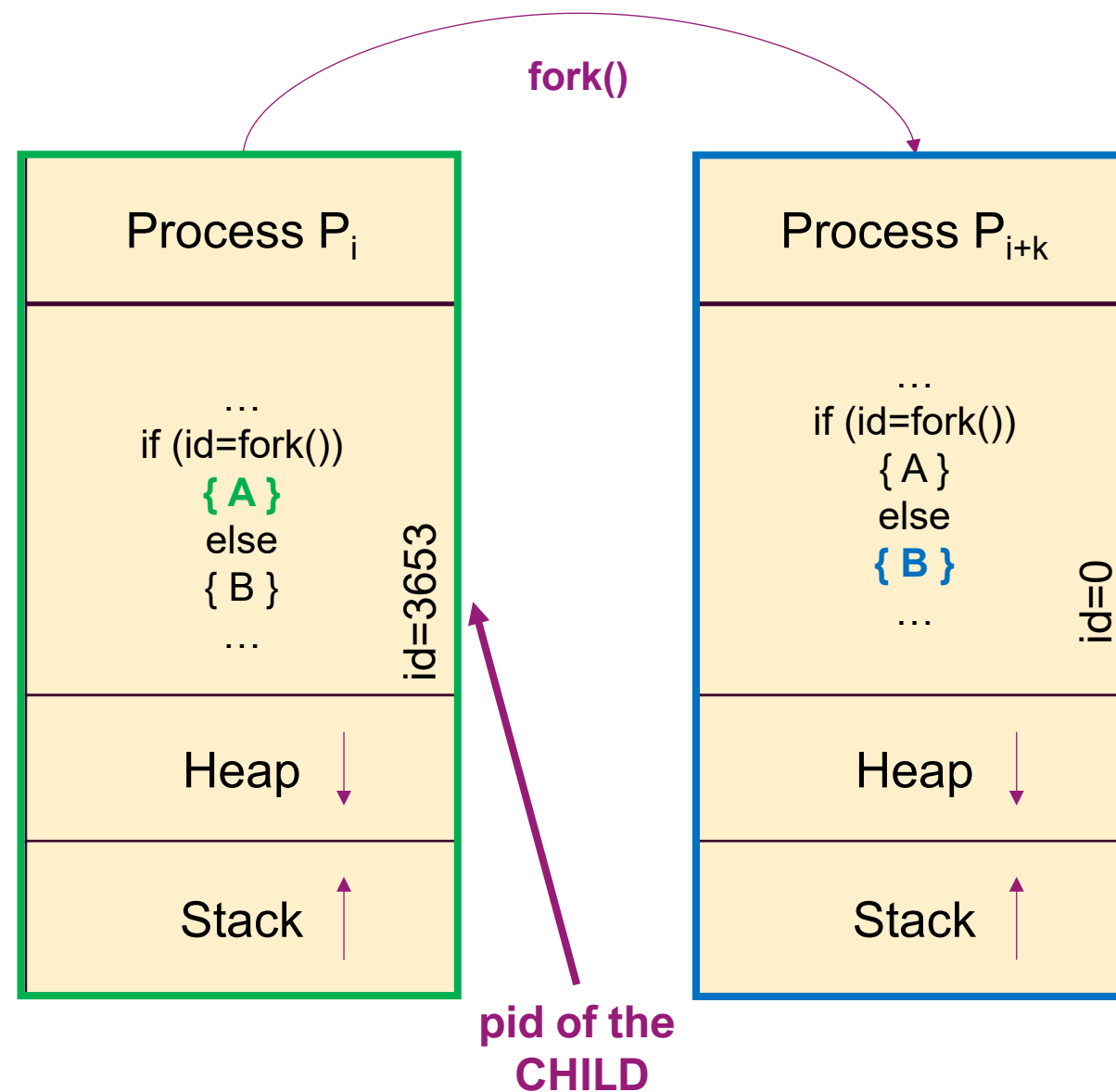
3) PROCESS

- Creation of a process
 - On Linux, a process is created by running the system function `fork()`
 - `fork()` creates two copies of the running process by copying its code



3) PROCESS

- Creation of a process
 - On Linux, a process is created by running the system function `fork()`
 - `fork()` creates two copies of the running process by copying its code
 - The two copies are exactly the same **except** the id number that is returned by the `fork` function:
 - **0** for the **CHILD**
 - **pid of the child** for the **PARENT**



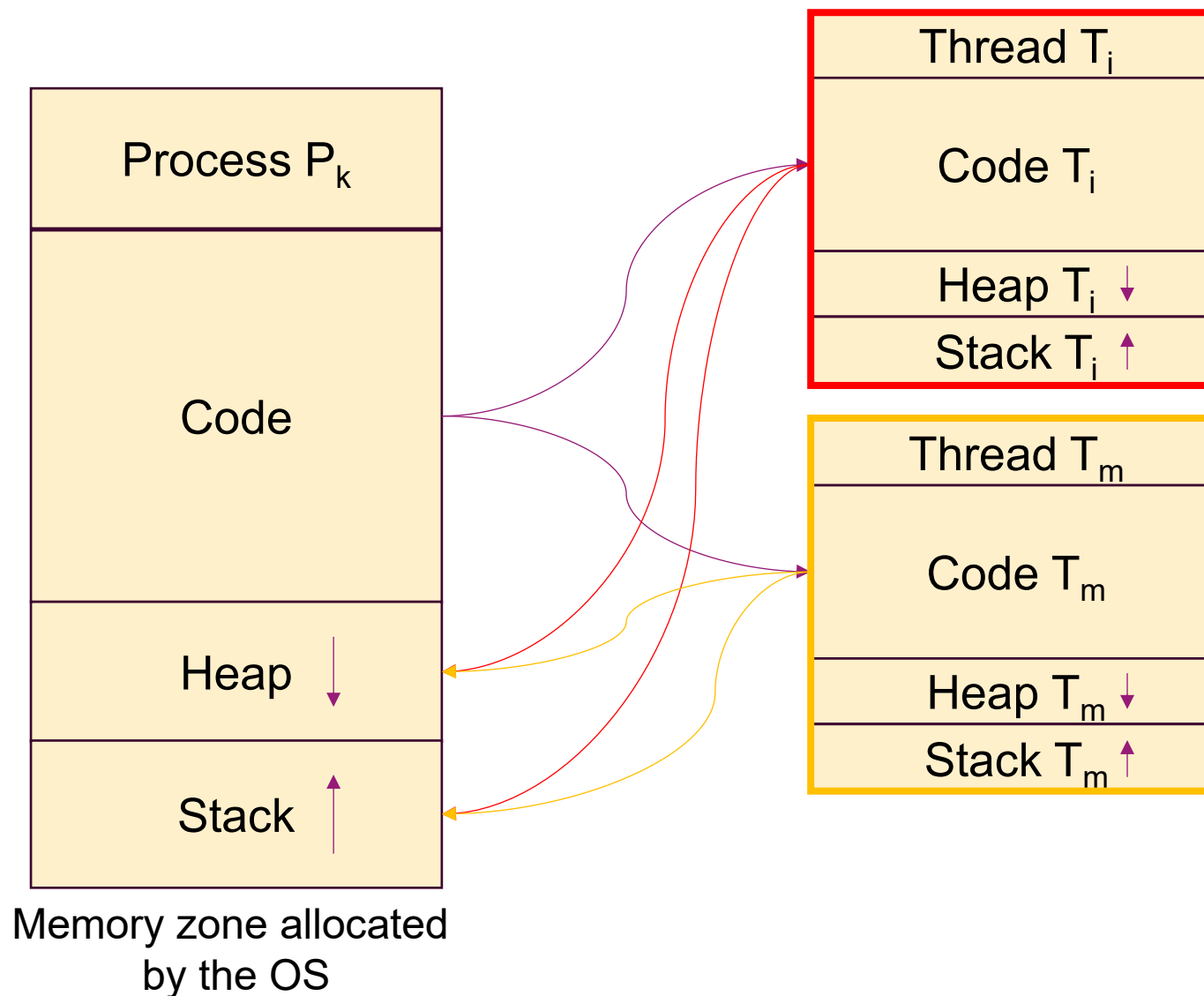
3) PROCESS

- Information in a process:

Process management	Memory management	File management
Registers Program counter Program status word Stack pointer Process state Priority Scheduling parameters Process ID Parent process Process group Signals Time when process started CPU time used Children's CPU time Time of next alarm	Pointer to text segment Pointer to data segment Pointer to stack segment	Root directory Working directory File descriptors User ID Group ID

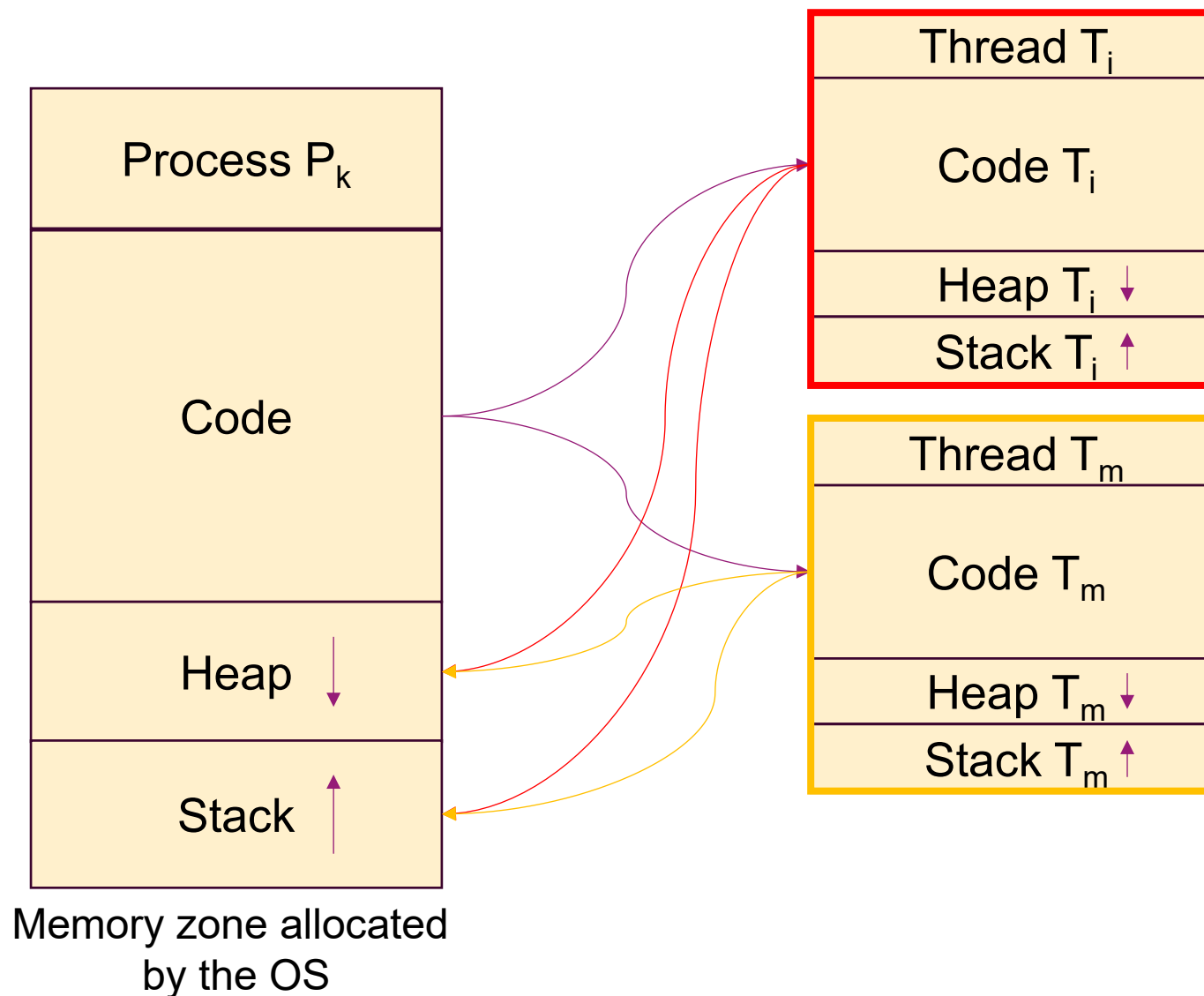
4) THREAD

- A thread is a light weight code running into a process



4) THREAD

- Creation of a thread:
 - On Linux, a thread is created using the **POSIX thread** interface by running the system function **pthread()**
 - **pthread()** creates two threads using the heap and stack of the process



4) THREAD

- Creation of a thread:
 - On Linux, a thread is created using the POSIX thread interface by running the system function `pthread()`
 - `pthread()` creates two threads using the heap and stack of the process

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <pthread.h>
4
5 volatile char theChar= '\0';
6 volatile char printTheChar=0;
7
8 void *readChar(void *name) {
9     do {
10         while (printTheChar==1);
11         theChar=getchar();
12         printTheChar=1;
13     } while (theChar!='Q');
14     return NULL;
15 }
16
17 void *displayChar (void *name) {
18     int counter=0;
19     do {
20         while (printTheChar==0) counter++;
21         printf("count: %d, char:%c\n", counter, theChar);
22         printTheChar=0;
23     } while (theChar!='Q');
24     return NULL;
25 }
26
27 int main(void) {
28     pthread_t childA, childB;
29
30     if (pthread_create(&childA, NULL, displayChar, "AAA")) {
31         perror("pthread_create\n"); exit(-1);
32     }
33     if (pthread_create(&childB, NULL, readChar, "BBB")) {
34         perror("pthread_create\n"); exit(-1);
35     }
36     if (pthread_join(childA, NULL))
37         perror("pthread_join A\n");
38     if (pthread_join(childB, NULL))
39         perror("pthread_join B\n");
40     printf("End of Process...\n");
41     return 0;
42 }
43 □
```

4) THREAD

- POSIX threads (pthread)

From Wikipedia, the free encyclopedia

POSIX Threads, commonly known as **pthreads**, is an [execution model](#) that exists independently from a language, as well as a parallel execution model. It allows a program to control multiple different flows of work that overlap in time. Each flow of work is referred to as a [thread](#), and creation and control over these flows is achieved by making calls to the POSIX Threads API. [POSIX](#) Threads is an [API](#) defined by the standard *POSIX.1c, Threads extensions (IEEE Std 1003.1c-1995)*.

The **Portable Operating System Interface (POSIX)** is a family of [standards](#) specified by the [IEEE Computer Society](#) for maintaining compatibility between [operating systems](#).

```

1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <pthread.h>
4
5 volatile char theChar= '\0';
6 volatile char printTheChar=0;
7
8 void *readChar(void *name) {
9     do {
10         while (printTheChar==1);
11         theChar=getchar();
12         printTheChar=1;
13     } while (theChar!='Q');
14     return NULL;
15 }
16
17 void *displayChar (void *name) {
18     int counter=0;
19     do {
20         while (printTheChar==0) counter++;
21         printf("count: %d, char:%c\n", counter, theChar);
22         printTheChar=0;
23     } while (theChar!='Q');
24     return NULL;
25 }
26
27 int main(void) {
28     pthread_t childA, childB;
29
30     if (pthread_create(&childA, NULL, displayChar, "AAA")) {
31         perror("pthread_create\n"); exit(-1);
32     }
33     if (pthread_create(&childB, NULL, readChar, "BBB")) {
34         perror("pthread_create\n"); exit(-1);
35     }
36     if (pthread_join(childA, NULL))
37         perror("pthread_join A\n");
38     if (pthread_join(childB, NULL))
39         perror("pthread_join B\n");
40     printf("End of Process...\n");
41     return 0;
42 }
43 □

```

```

[jlandre@jlandre-LINUX code]$ gcc -o pthread-00 pthread-00.c
[jlandre@jlandre-LINUX code]$ ./pthread-00
a
count: 407992937, char:a
count: 407993067, char:

b
count: 515353073, char:b
count: 515353209, char:

c
count: 625600908, char:c
count: 625601049, char:

Q
count: 863138219, char:Q
End of Process...
[jlandre@jlandre-LINUX code]$ □

```

4) THREAD

- Shared and private resources for threads:

Per process items	Per thread items
Address space	Program counter
Global variables	Registers
Open files	Stack
Child processes	State
Pending alarms	
Signals and signal handlers	
Accounting information	

Source: (Tanenbaum&Bos, "Modern OperatingSystems, fifth edition", Pearson, 2024)

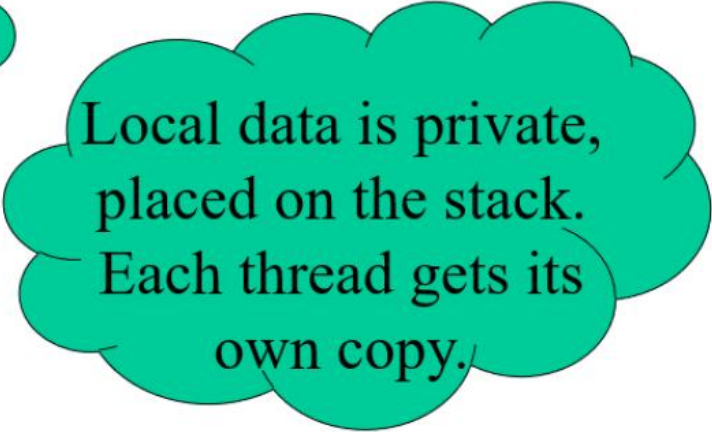
4) THREAD

- Shared and private data for threads:

```
int glob;
void main()
{ int lok;
  // ...
}
```



Global data shared
between threads.



Local data is private,
placed on the stack.
Each thread gets its
own copy.

5) COMPARISON BETWEEN PROCESSES AND THREADS

Process

A process is heavyweight.

A process has its own memory.

Inter-process communication is slower due to isolated memory.

Context switching between processes is expensive due to saving old and loading new process memory and stack info.

An application with several processes for its components can provide better memory utilization when memory is scarce. We can assign low priority to inactive processes in the application. This idle process is then eligible to be swapped to disk. This keeps the active components of the application responsive.

Thread

A thread is a lightweight process also called an LWP.

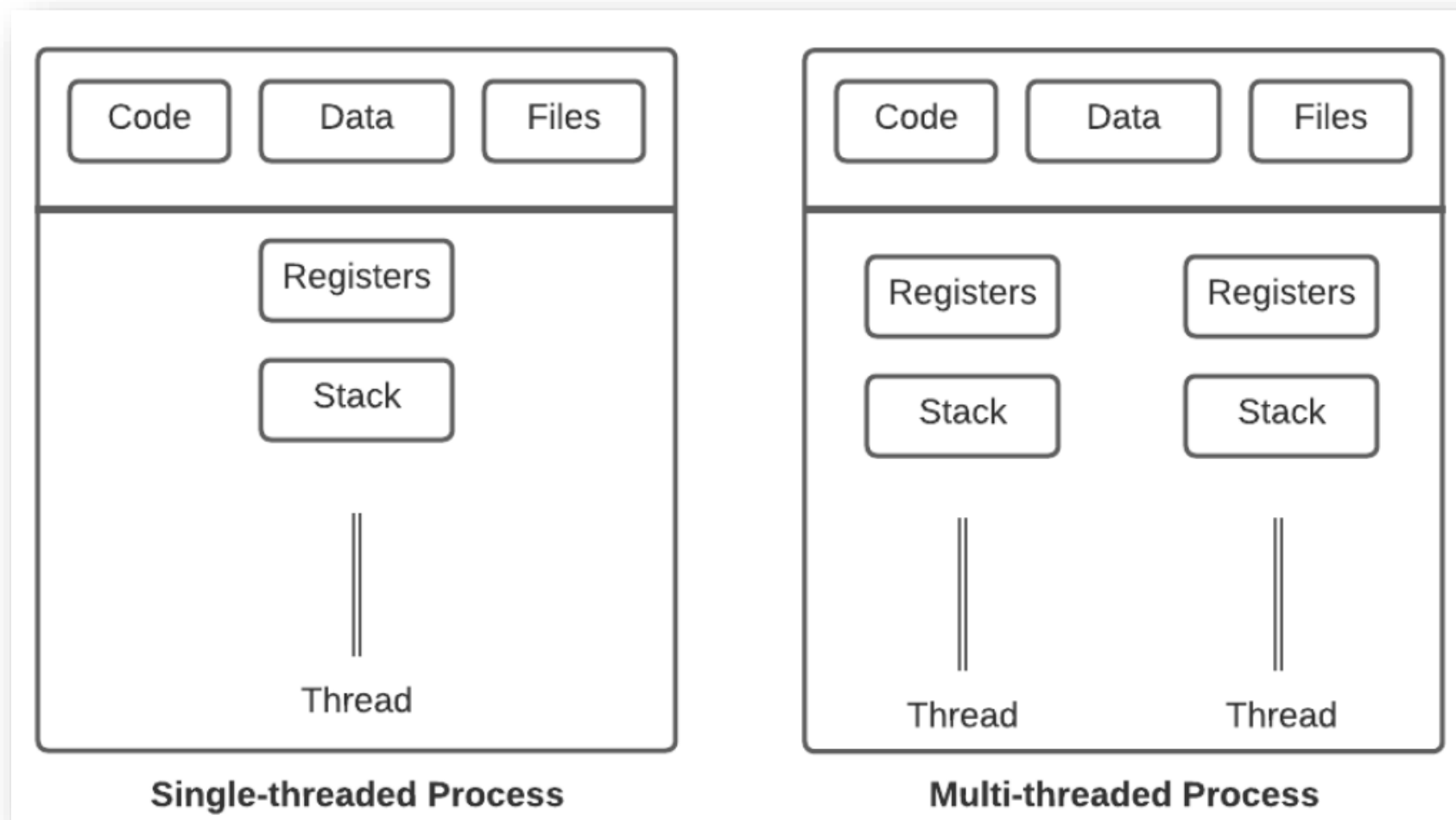
A thread shares the memory with the parent process and other threads within the process.

Inter-thread communication is faster due to shared memory.

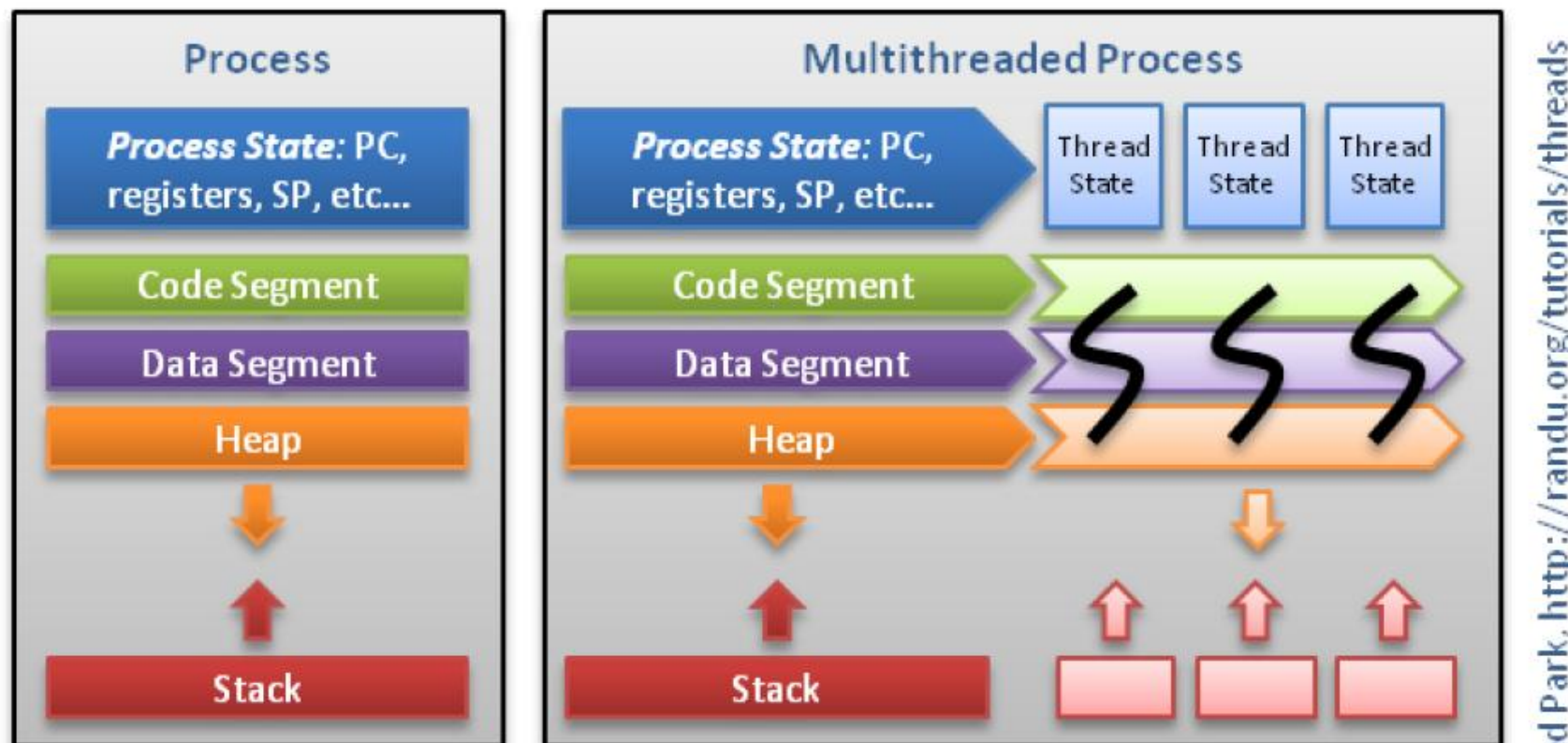
Context switching between threads is less expensive due to shared memory.

When memory is scarce, the multi-threaded application does not provide any provision to manage memory.

5) COMPARISON BETWEEN PROCESSES AND THREADS



5) COMPARISON BETWEEN PROCESSES AND THREADS



Threads contain only necessary information, such as a stack (for local variables, function arguments, return values), a copy of the registers, program counter and any thread-specific data to allow them to be scheduled individually. Other data is shared within the process between all threads.

© Alfred Park, <http://randu.org/tutorials/threads>

task_struct Struct Reference

Data Fields

#include <sched.h>

Data Fields

volatile long	state
void *	stack
atomic_t	usage
unsigned int	flags
unsigned int	ptrace
int	on_rq
int	prio
int	static_prio
int	normal_prio
unsigned int	rt_priority
struct sched_class *	sched_class
struct sched_entity	se
struct sched_rt_entity	rt
unsigned char	fpu_counter
unsigned int	policy
int	nr_cpus_allowed
cpumask_t	cpus_allowed
struct list_head	tasks
struct mm_struct *	mm
struct mm_struct *	active_mm
int	exit_state
int	exit_code
int	exit_signal
int	pdeath_signal
unsigned int	jobctl
unsigned int	personality
unsigned	did_exec:1
unsigned	in_execve:1
unsigned	in_iowait:1
unsigned	no_new_privs:1
unsigned	sched_reset_on_fork:1
unsigned	sched_contributes_to_load:1
pid_t	pid
pid_t	tgid
struct task_struct __rcu *	real_parent
struct task_struct __rcu *	parent
struct list_head	children

6) TREE OF PROCESSES

- Because `fork()` create two copies of a process each time it is launched, the processes are organised as a **tree**

\$ **pstree**

```
[jlandre@jlandre-LINUX code]$ pstree
systemd--NetworkManager--3*[{NetworkManager}]
      bluetoothd
      dbus-daemon
      firewalld--[{firewalld}]
      irqbalance--[{irqbalance}]
      kwalletd5--4*[{kwalletd5}]
      polkitd--3*[{polkitd}]
      power-profiles--3*[{power-profiles-}]
      sddm--Xorg--3*[{Xorg}]
            sddm-helper--startplasma-way--[{startplasma-way}]
            [{sddm}]
      sshd
      systemd--(sd-pam)
            at-spi-bus-laun--dbus-daemon
            at-spi2-registr--3*[{at-spi2-registr}]
            baloo_file--3*[{baloo_file}]
            dbus-daemon
            dconf-service--3*[{dconf-service}]
            firewall-applet--7*[{firewall-applet}]
            gmenudbusmenupr--2*[{gmenudbusmenupr}]
            kaccess--3*[{kaccess}]
            kactivitymanage--6*[{kactivitymanage}]
            kded5--10*[{kded5}]
            kioslave5--2*[{kioslave5}]
            kioslave5--2*[{kioslave5}]
            kioslave5--8*[{kioslave5}]
            ksmserver--3*[{ksmserver}]
            kwin_wayland_wr--kwin_wayland--Xwayland--[{Xwayland}]
            [{kwin_wayland_wr}]
            matray--4*[{matray}]
            obexd
            org_kde_powerde--8*[{org_kde_powerde}]
            pamac-tray-plas--6*[{pamac-tray-plas}]
            pipewire--2*[{pipewire}]
            pipewire-pulse--2*[{pipewire-pulse}]
            plasmashell--konsole--bash--pstree
            [{konsole}]
            polkit-kde-auth--7*[{polkit-kde-auth}]
            spectacle--13*[{spectacle}]
            wireplumber--6*[{wireplumber}]
            xembedsniproxy--2*[{xembedsniproxy}]
      systemd-homed
      systemd-journal
      systemd-logind
      systemd-network
      systemd-resolve
      systemd-timesyn--[{systemd-timesyn}]
      systemd-udev
      systemd-userdbd--3*[{systemd-userwor}]
      udisksd--5*[{udisksd}]
      upowerd--3*[{upowerd}]
      wpa_supplicant
[jlandre@jlandre-LINUX code]$
```

6) TREE OF PROCESSES

- When we create processes, we build a **tree** of processes

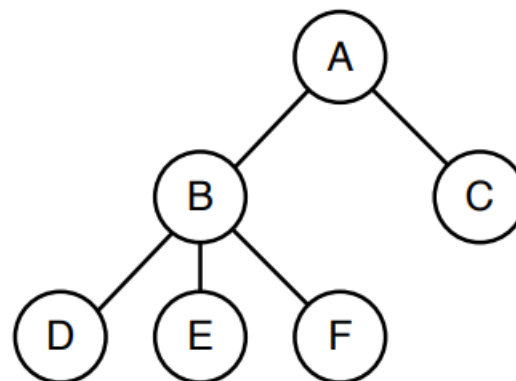


Figure 1-13. A process tree. Process *A* created two child processes, *B* and *C*. Process *B* created three child processes, *D*, *E*, and *F*.

6) TREE OF PROCESSES

- Processes and threads ids can be seen with:

\$ ps -eLf

```
[jlandre@jlandre-LINUX code]$ ps -eLf
UID      PID     PPID    LWP   C  NLWP  STIME TTY      TIME CMD
root         1         0      1   0     1  18:04 ?        00:00:02 /sbin/init splash
root         2         0      2   0     1  18:04 ?        00:00:00 [kthreadd]
root         3         2      3   0     1  18:04 ?        00:00:00 [rcu_gp]
root         4         2      4   0     1  18:04 ?        00:00:00 [rcu_par_gp]
root         5         2      5   0     1  18:04 ?        00:00:00 [slub_flushwq]
root         6         2      6   0     1  18:04 ?        00:00:00 [netns]
root         7         2      7   0     1  18:04 ?        00:00:00 [kworker/0:0-events]
root        10         2     10   0     1  18:04 ?        00:00:00 [mm_percpu_wq]
root        11         2     11   0     1  18:04 ?        00:00:00 [rcu_tasks_kthread]
root        12         2     12   0     1  18:04 ?        00:00:00 [rcu_tasks_rude_kthread]
root        13         2     13   0     1  18:04 ?        00:00:00 [rcu_tasks_trace_kthread]
root        14         2     14   0     1  18:04 ?        00:00:00 [ksoftirqd/0]
root        15         2     15   0     1  18:04 ?        00:00:01 [rcu_preempt]
root        16         2     16   0     1  18:04 ?        00:00:00 [migration/0]
root        17         2     17   0     1  18:04 ?        00:00:00 [cpuhp/0]
root        18         2     18   0     1  18:04 ?        00:00:00 [cpuhp/1]
root        19         2     19   0     1  18:04 ?        00:00:00 [migration/1]
root        20         2     20   0     1  18:04 ?        00:00:00 [ksoftirqd/1]
root        23         2     23   0     1  18:04 ?        00:00:00 [cpuhp/2]
```

6) TREE OF PROCESSES

UID PID PPID **LWP** ...

[illegible]

ANY QUESTIONS?